

Artificial Neural Network (ANN)

Gradient Descent

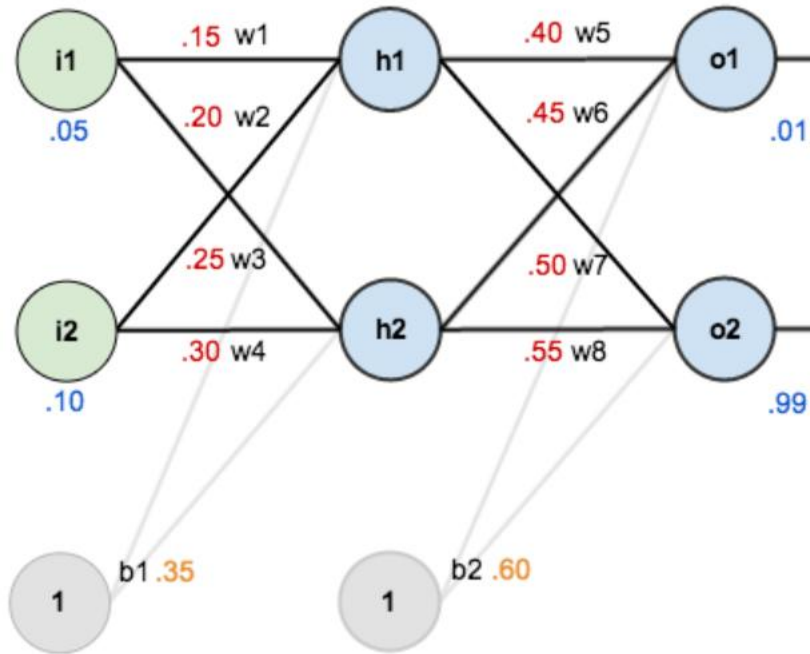
One Hot Encoding:

It converts categorical values into binary vectors (0s and 1s), where only one position is **1** and the rest are **0**.

Color = {Red, Green, Blue}

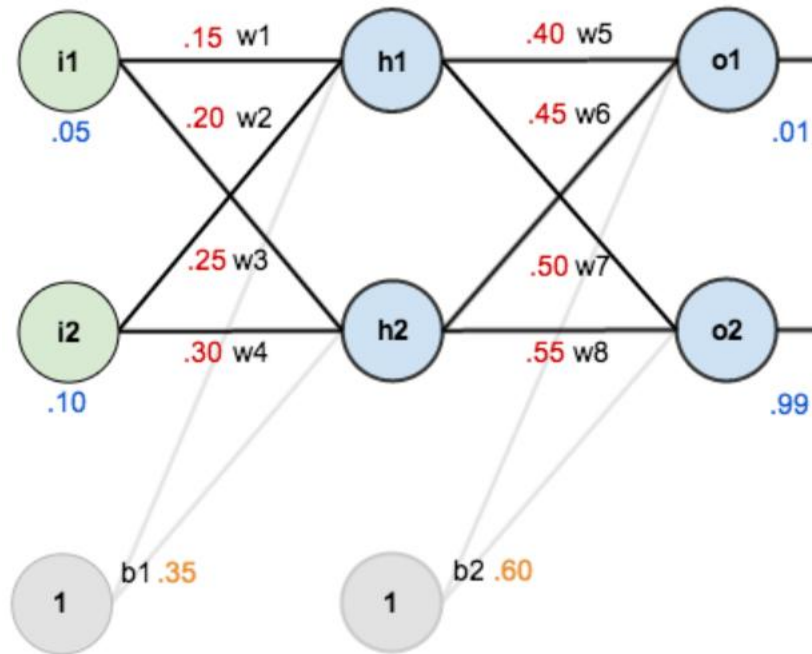
Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

Train ANN Example



- **Inputs:** $i_1 = 0.05, i_2 = 0.10$
- **Targets:** $target_{o1} = 0.01, target_{o2} = 0.99$
- **Biases:** $b_1 = 0.35, b_2 = 0.60$
- **Learning Rate (η):** 0.5
- **Activation:** Logistic function $out = \frac{1}{1+e^{-net}}$

Forward Pass



Hidden Layer

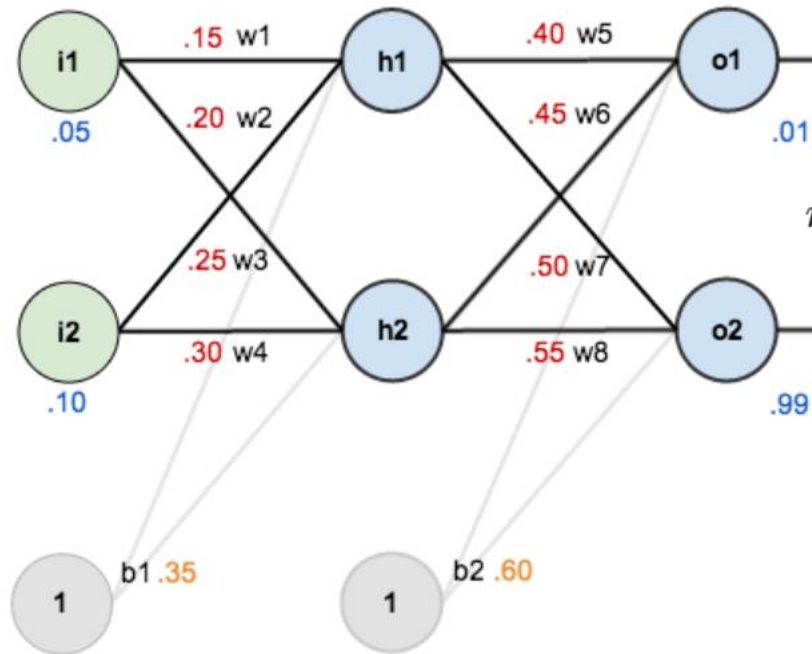
At neuron h1:

$$net_{h1} = w_1 \cdot i_1 + w_2 \cdot i_2 + b_1 \cdot 1$$

$$net_{h1} = (0.15 \cdot 0.05) + (0.20 \cdot 0.10) + 0.35 = 0.0075 + 0.02 + 0.35 = \mathbf{0.3775}$$

$$out_{h1} = \frac{1}{1 + e^{-net_{h1}}} = \frac{1}{1 + e^{-0.3775}} \approx \mathbf{0.59326999}$$

Forward Pass



Hidden Layer

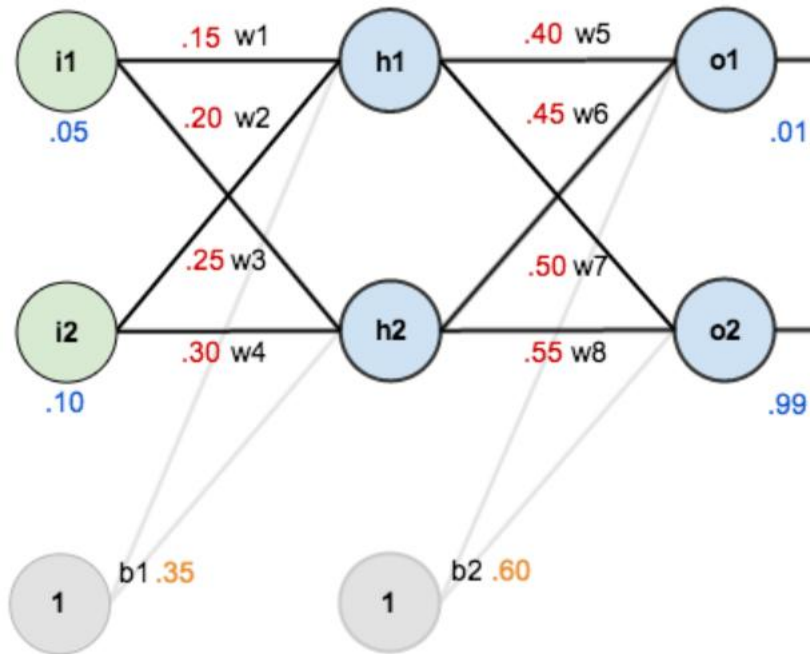
At neuron h2:

$$net_{h2} = w_3 \cdot i_1 + w_4 \cdot i_2 + b_1 \cdot 1$$

$$net_{h2} = (0.25 \cdot 0.05) + (0.30 \cdot 0.10) + 0.35 = 0.0125 + 0.03 + 0.35 = \mathbf{0.3925}$$

$$out_{h2} = \frac{1}{1 + e^{-net_{h2}}} = \frac{1}{1 + e^{-0.3925}} \approx \mathbf{0.59688438}$$

Forward Pass



Output Layer

At neuron o1:

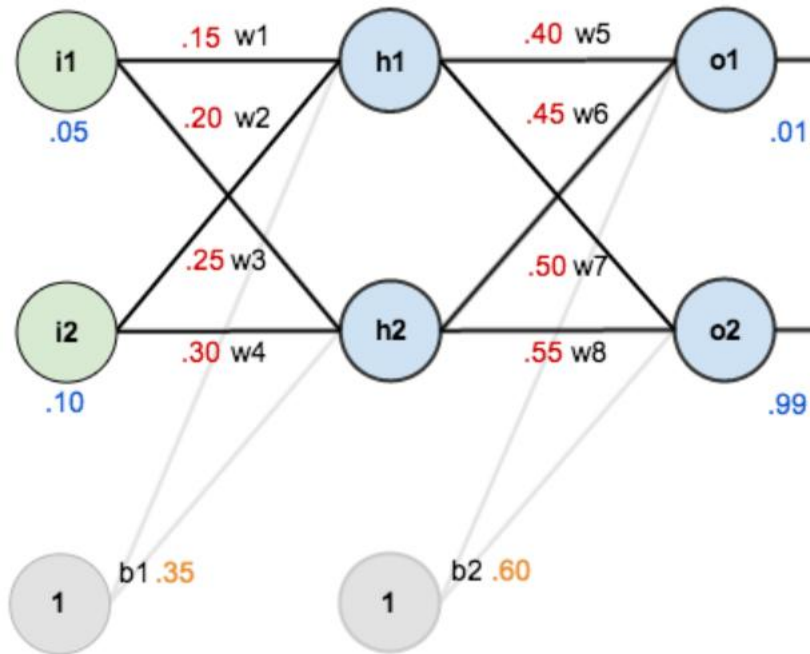
$$net_{o1} = w_5 \cdot out_{h1} + w_6 \cdot out_{h2} + b_2 \cdot 1$$

$$net_{o1} = (0.40 \cdot 0.59326999) + (0.45 \cdot 0.59688438) + 0.60$$

$$net_{o1} = 0.23730800 + 0.26859797 + 0.60 = \mathbf{1.10590597}$$

$$out_{o1} = \frac{1}{1 + e^{-1.10590597}} \approx \mathbf{0.75136507}$$

Forward Pass



Output Layer

At neuron o2:

$$net_{o2} = w_7 \cdot out_{h1} + w_8 \cdot out_{h2} + b_2 \cdot 1$$

$$net_{o2} = (0.50 \cdot 0.59326999) + (0.55 \cdot 0.59688438) + 0.60$$

$$net_{o2} = 0.29663500 + 0.32828641 + 0.60 = \mathbf{1.22492141}$$

$$out_{o2} = \frac{1}{1 + e^{-1.22492141}} \approx \mathbf{0.77292847}$$

Forward Pass

Error Calculation

$$out_{o2} = \frac{1}{1 + e^{-1.22492141}} \approx \mathbf{0.77292847}$$

$$out_{o1} = \frac{1}{1 + e^{-1.10590597}} \approx \mathbf{0.75136507}$$

$$E = \frac{1}{2}(target - out)^2$$

Error at o1:

$$E_{o1} = \frac{1}{2}(0.01 - 0.75136507)^2$$

$$E_{o1} = \frac{1}{2}(-0.74136507)^2 \approx \mathbf{0.27481108}$$

Error at o2:

$$E_{o2} = \frac{1}{2}(0.99 - 0.77292847)^2$$

$$E_{o2} = \frac{1}{2}(0.21707153)^2 \approx \mathbf{0.02356003}$$

Total Error :

$$E_{total} = E_{o1} + E_{o2}$$

$$E_{total} = 0.27481108 + 0.02356003 = \mathbf{0.29837111}$$

Back Propagation

Formulas

Use Chain Rule.

$$\overset{1}{\frac{\partial E_{total}}{\partial w_5}} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1})$$

Back Propagation

Formulas

Use Chain Rule.

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

=

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1})$$

Back Propagation

Formulas

Use Chain Rule.

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5} \quad \textcircled{3}$$

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1}$$

Back Propagation

Output Layer

Use Chain Rule.

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

Combine them all:

$$\frac{\partial E_{total}}{\partial w_5} = -(target_{o1} - out_{o1}) * out_{o1}(1 - out_{o1}) * out_{h1}$$

$$\delta_{o1} = -(target_{o1} - out_{o1}) * out_{o1}(1 - out_{o1})$$

Similarly, Find Sigma value for o2?

Back Propagation

Hidden Layer

Use Chain Rule.

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

Equivalent to:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial E_{total}}{\partial w}$$

$$\frac{\partial E_{total}}{\partial w_1} = \left(\sum_o \frac{\partial E_{total}}{\partial out_o} * \frac{\partial out_o}{\partial net_o} * \frac{\partial net_o}{\partial out_{h1}} \right) * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = \left(\sum_o \delta_o \cdot w_{ho} \right) \cdot out_{h1} (1 - out_{h1}) \cdot i_1$$

Intuition: Find Sigma value of hidden values w.r.t output layers.

Back Propagation

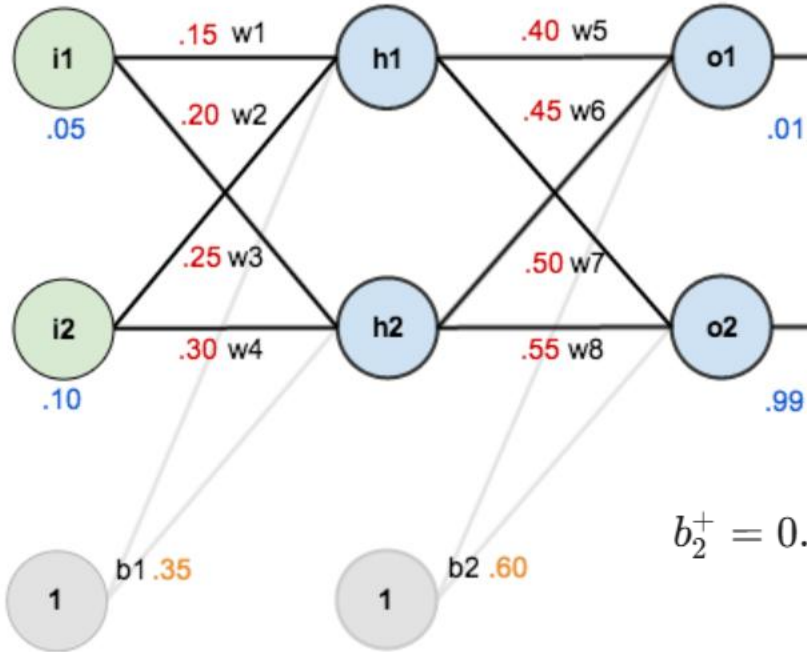
$$b_2^+ = b_2 - \eta \times (\delta_{o1} + \delta_{o2})$$

$$b_1^+ = b_1 - \eta \times (\delta_{h1} + \delta_{h2})$$

Bias Calculation

Weight Updates:

Find δ at Output Layer



$$\delta_{o1} = \frac{\partial E_{total}}{\partial net_{o1}} = -(target_{o1} - out_{o1}) \times out_{o1}(1 - out_{o1})$$

$$\delta_{o2} = \frac{\partial E_{total}}{\partial net_{o2}} = -(target_{o2} - out_{o2}) \times out_{o2}(1 - out_{o2})$$

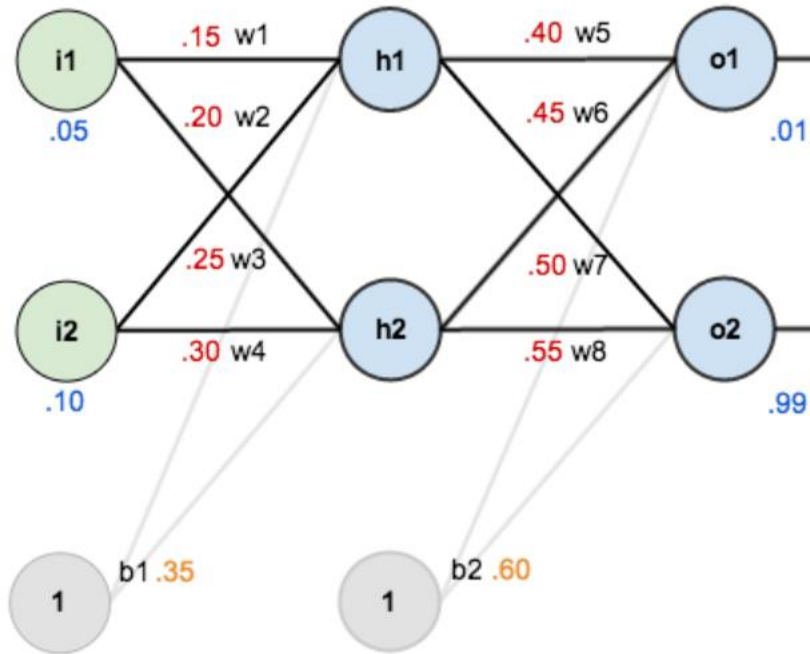
$$\delta_{o1}: -(0.01 - 0.75136507) \times 0.75136507(1 - 0.75136507) \approx \mathbf{0.13849856}$$

$$\delta_{o2}: -(0.99 - 0.77292846) \times 0.77292846(1 - 0.77292846) \approx \mathbf{-0.03809824}$$

$$b_2^+ = 0.60 - 0.5 \times (0.13849856 - 0.03809824) = 0.60 - 0.05020016 = \mathbf{0.54979984}$$

$$E_{total} = 0.27481108 + 0.02356003 = \mathbf{0.29837111}$$

Weight Updates:



$$E_{total} = 0.27481108 + 0.02356003 = \mathbf{0.29837111}$$

Find δ at Hidden Layer

$$\delta_{h1} = \left(\sum \delta_o \times w_{corresponding} \right) \times out_{h1}(1 - out_{h1})$$

$$\delta_{h1} = (\delta_{o1} \cdot w_5 + \delta_{o2} \cdot w_7) \times out_{h1}(1 - out_{h1})$$

$$\delta_{h2} = (\delta_{o1} \cdot w_6 + \delta_{o2} \cdot w_8) \times out_{h2}(1 - out_{h2})$$

$$\delta_{h1}: (0.13849856 \cdot 0.4 + (-0.03809824) \cdot 0.5) \times 0.59326999(1 - 0.59326999) \approx \mathbf{0.00871385}$$

$$\delta_{h2}: (0.13849856 \cdot 0.45 + (-0.03809824) \cdot 0.55) \times 0.59688438(1 - 0.59688438) \approx \mathbf{0.00995023}$$

$$b_1^+ = 0.35 - 0.5 \times (0.00871385 + 0.00995023) = 0.35 - 0.00933204 = \mathbf{0.34066796}$$

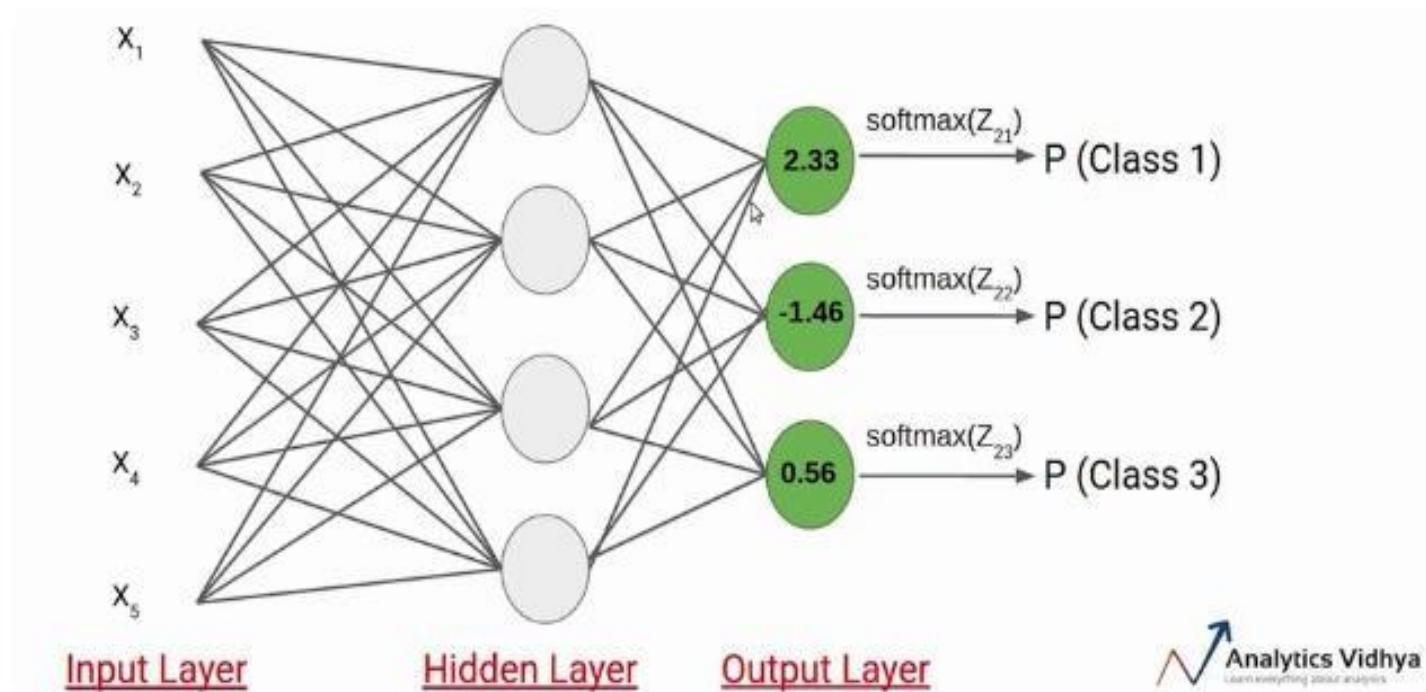
Updated Weights are:

Weight	Formula	Updated Value
w_5	$w_5 - \eta \cdot \delta_{o1} \cdot out_{h1}$	0.35891648
w_6	$w_6 - \eta \cdot \delta_{o1} \cdot out_{h2}$	0.40866619
w_7	$w_7 - \eta \cdot \delta_{o2} \cdot out_{h1}$	0.51130127
w_8	$w_8 - \eta \cdot \delta_{o2} \cdot out_{h2}$	0.56137012
w_1	$w_1 - \eta \cdot \delta_{h1} \cdot i_1$	0.14978215
w_2	$w_2 - \eta \cdot \delta_{h1} \cdot i_2$	0.19956431
w_3	$w_3 - \eta \cdot \delta_{h2} \cdot i_1$	0.24975124
w_4	$w_4 - \eta \cdot \delta_{h2} \cdot i_2$	0.29950249

Verify these please?

Softmax Activation Function:

Suppose the value of Z_{21} , Z_{22} , Z_{23} comes out to be 2.33, -1.46, and 0.56 respectively. Now the SoftMax activation function is applied to each of these neurons and the following values are generated.



Softmax Activation Function:

These are the probability values that a data point belonging to the respective classes. Note that, the sum of the probabilities, in this case, is equal to 1.

Example :

$$\text{2.33} \rightarrow P(\text{Class 1}) = \frac{\exp(2.33)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.83827314$$

$$\text{-1.46} \rightarrow P(\text{Class 2}) = \frac{\exp(-1.46)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.01894129$$

$$\text{0.56} \rightarrow P(\text{Class 3}) = \frac{\exp(0.56)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.14278557$$

Train ANN

Training neural nets:

Loop until convergence:

► for each example n

1. Given input $\mathbf{x}^{(n)}$, propagate activity forward ($\mathbf{x}^{(n)} \rightarrow \mathbf{h}^{(n)} \rightarrow \mathbf{o}^{(n)}$)
(**forward pass**)
2. Propagate gradients backward (**backward pass**)
3. Update each weight (via gradient descent)